INDUSTRIAL SOFTWARE ENGINEERING RELEASE 1 DOCUMENT

Date: 22 March 2026
Project Title: JanVaani AI — Smart Public Grievance System
Team Members: Mangam Rajvardhan, Paidipati Gurudeep, Prince Kumar.

1. PROBLEM STATEMENT

Citizens often face significant challenges in reporting public grievances efficiently due to the absence of streamlined and user-friendly systems. One of the primary issues is the lack of proper routing mechanisms, which leads to delays in resolving complaints as they may not reach the appropriate departments in time.

Existing platforms are frequently complex, non-intuitive, and do not provide adequate multilingual support, making them inaccessible to a large segment of the population. Additionally, many users struggle to clearly articulate their complaints in structured formats, further complicating the process.

There is also no intelligent system in place to automatically classify complaints and direct them to the correct authorities. As a result, these limitations contribute to delayed governance responses, reduced citizen engagement, and the overall inefficient handling of public grievances.

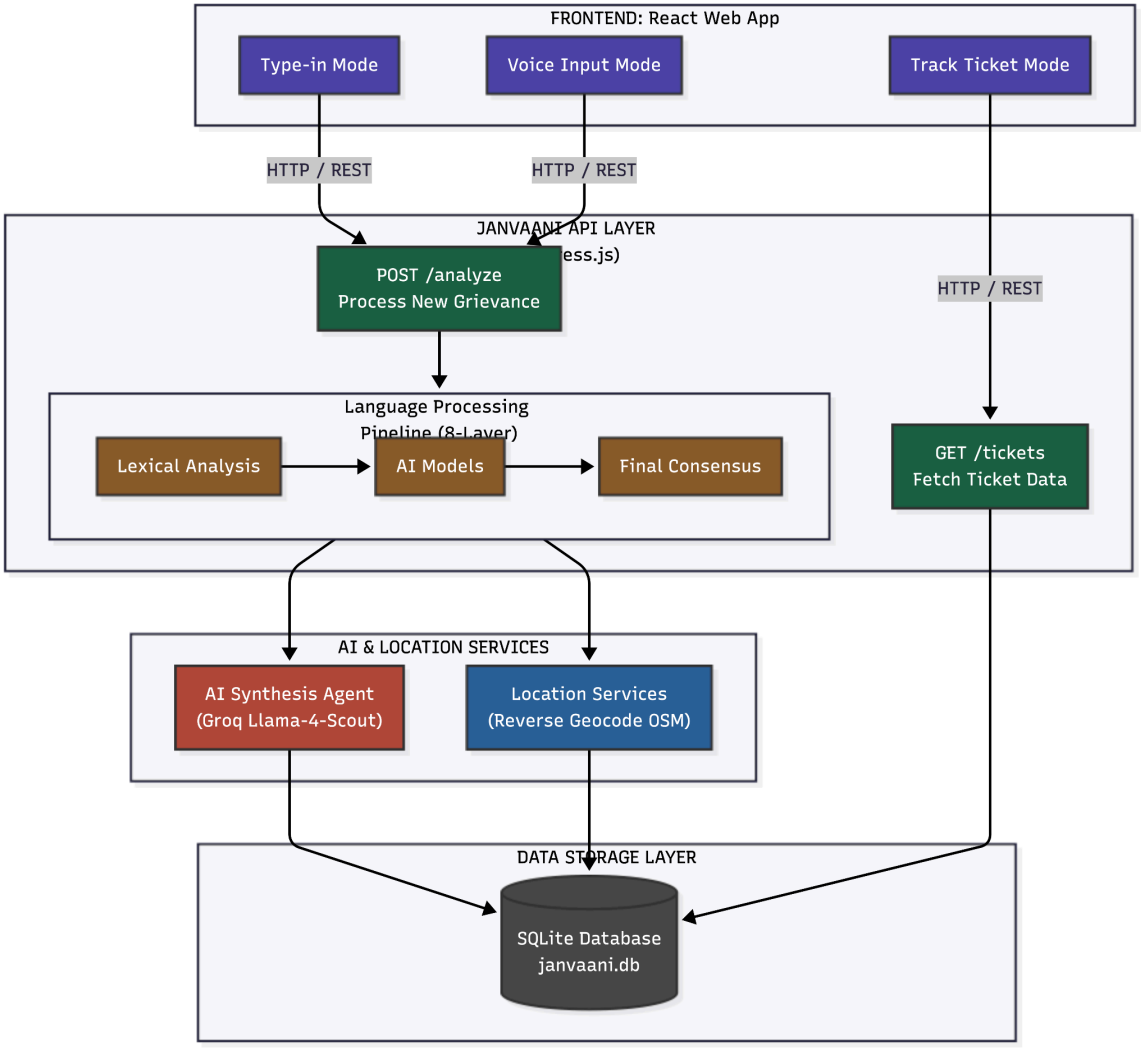
2. PROPOSED SOLUTION

The proposed solution is to develop an AI-powered grievance submission platform called *JanVaani AI*, designed to make the complaint process simple, efficient, and accessible to all citizens. The platform will support both voice and text-based inputs, enabling users to submit grievances in a natural and convenient manner. It will incorporate automatic language handling to cater to users from diverse linguistic backgrounds.

Using AI-based analysis, the system will refine and structure complaints for better clarity, while also intelligently classifying them into the appropriate departments. Additionally, location-based routing using geospatial logic will ensure that complaints are directed to the correct local authorities.

This solution aims to simplify the complaint submission process, reduce ambiguity in grievance descriptions, and ensure accurate department routing, especially benefiting non-technical users. Ultimately, it is expected to lead to faster grievance resolution, increased citizen participation, and improved overall governance efficiency.

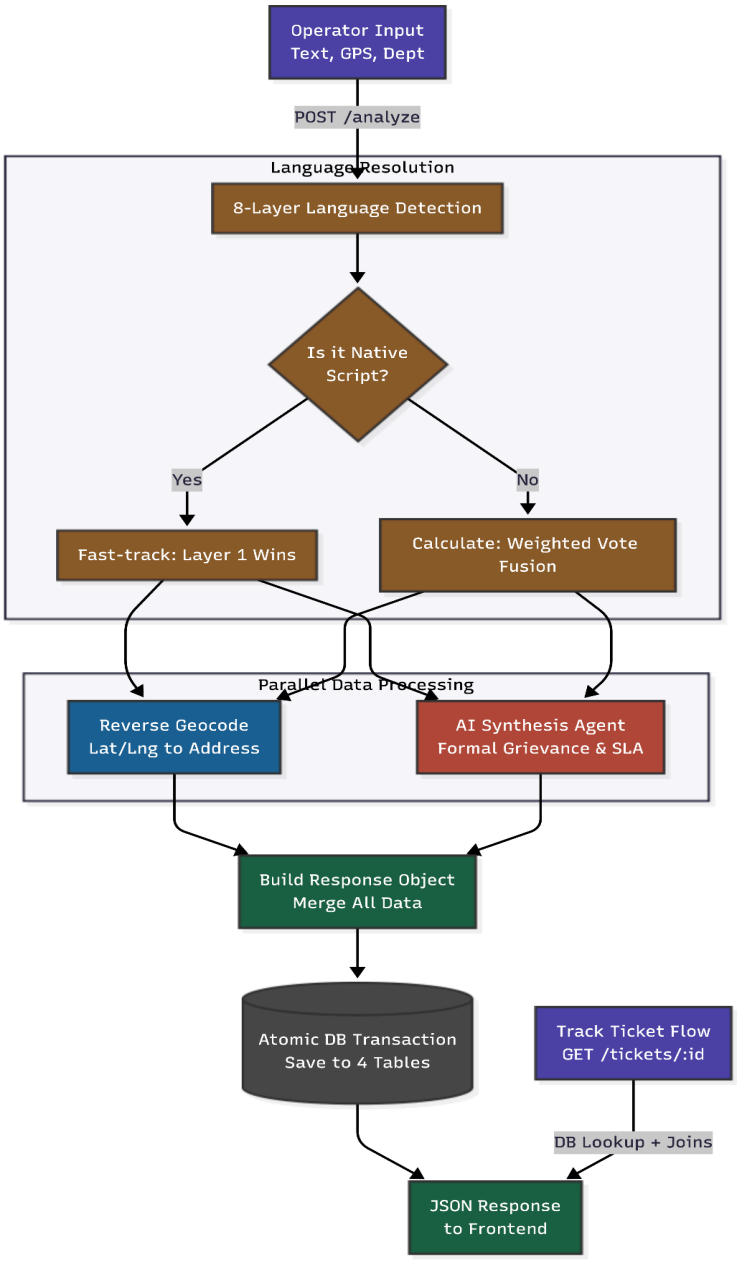
3. SYSTEM ARCHITECTURE



The architecture of the JanVaani AI system follows a deeply integrated, decoupled client-server model engineered for multi-modal grievance processing. The presentation layer is a React-based Single Page Application (SPA) that functions as an operator portal, capturing inputs through manual text entry, the browser's Geolocation API, and real-time voice dictation via the Web Speech API. This frontend communicates via RESTful HTTP requests with a Node.js/Express backend that acts as the primary orchestration and routing hub. Upon receiving a payload at the `/analyze` endpoint, the backend triggers a highly concurrent data enrichment workflow. It translates raw GPS coordinates into structured addresses using the Nominatim OpenStreetMap API, while simultaneously routing the grievance text through a proprietary 8-layer language detection engine. This engine calculates a weighted consensus by fusing local deterministic rules (Unicode blocks, N-grams, custom dictionaries, and phonetic heuristics) with statistical NLP (Franc) and cloud-based AI inference via Hugging Face (XLM-RoBERTa) and

Groq (Llama 3.1, 3.3, and 4-Scout). Once the linguistic context is established, an AI Synthesis Agent powered by Groq's Llama-4-Scout model classifies the grievance, calculates severity-based SLAs mapped to a localized department registry, and generates formal documentation. Finally, all synthesized data, along with granular language detection votes and word-level matches, is securely committed to a local SQLite database utilizing strict atomic transactions and Write-Ahead Logging (WAL) to guarantee relational data integrity.

- Data Flow Diagram



The data flow initiates when an operator captures a citizen's grievance through the React Single Page Application (SPA), utilizing either manual text entry or real-time voice dictation via the Web Speech API, alongside essential metadata like GPS coordinates. This payload is securely transmitted as a JSON request to the Express.js backend via the `POST /analyze` endpoint. Upon receipt, the backend orchestrates a parallel processing workflow: it reverse-geocodes the location coordinates using Nominatim OpenStreetMap, while simultaneously routing the raw text through a proprietary 8-layer language detection pipeline. This pipeline calculates a weighted consensus by blending local deterministic rules with advanced inference calls to the Hugging Face API (XLM-RoBERTa) and Groq's Llama models. Once the exact linguistic context is resolved, the data is handed off to the AI Synthesis Agent (powered by Groq Llama-4-Scout), which intelligently determines the appropriate government department, assigns a severity tier, drafts a formal grievance, and calculates strict SLA deadlines based on a local registry. This fully enriched dataset is then safely committed to a local SQLite database (`janvaani.db`) executing a single, atomic transaction across four normalized tables to ensure data integrity. Finally, the backend dispatches a structured JSON response to the frontend, rendering a comprehensive result card that displays the newly generated Ticket ID, SLA commitments, and a localized script for the operator to read back to the citizen.

4. TECH STACK

- Frontend: React

  The operator portal is built as a React Single Page Application (SPA). This component-driven architecture facilitates a highly responsive, multi-modal interface, allowing operators to seamlessly switch between manual typing, ticket tracking, and real-time voice dictation using the native Web Speech API. It also securely captures precise citizen coordinates via the browser's Geolocation API.

- Backend: Node.js + Express

  The core orchestration layer is powered by Node.js and Express.js. This lightweight, event-driven framework efficiently manages RESTful HTTP routing, handles Cross-Origin Resource Sharing (CORS), and coordinates complex, concurrent tasks—such as executing the 8-layer language detection pipeline and routing data to external AI services—without blocking the main thread..

- Database: SQLite (better-sqlite3)

  Unlike earlier phases, data persistence is actively implemented using a highly

performant local SQLite database. The system employs Write-Ahead Logging (WAL) and strict atomic transactions across four normalized tables (`tickets`, `ticket_updates`, `language_votes`, `word_detections`) to ensure the immediate, secure, and relational storage of grievance records and audit trails.

- AI / ML: Groq SDK, Hugging Face & Custom NLP

  The platform's intelligence relies on a hybrid AI approach. It integrates the Groq API (utilizing models like Llama-4-Scout) to act as the primary Synthesis Agent, responsible for grievance structuring, severity classification, and SLA generation. Furthermore, it leverages the Hugging Face Inference API (XLM-RoBERTa) alongside statistical models (Franc) and custom lexical dictionaries to power its robust, 8-layer script and language consensus engine.

- DevOps / Hosting: Planned (Docker / Cloud)

  DevOps and hosting infrastructure are planned for future implementation using Docker and cloud platforms. Currently, the system is running in a local development environment, with plans to enable scalable and reliable deployment.

- Collaboration Tools: GitHub

  GitHub is used for version control and collaboration, allowing team members to manage code changes, track contributions, and maintain a structured development workflow.

5. TEAM ROLES

- P Gurudeep : Backend Developer

  Working as the Backend Developer, where he was responsible for architecting and implementing the Express-based server infrastructure. He effectively integrated the Groq API to enable AI-driven functionalities and developed robust routing logic to ensure accurate mapping of complaints to the appropriate departments. He also played a pivotal role in designing and structuring the overall system architecture.

- M Rajvardhan : Frontend Developer

  Working as the Frontend Developer, leading the development of a responsive and user-centric interface using React. He managed application state and user workflows to ensure a seamless and intuitive user experience, implemented efficient API integrations for frontend-backend communication, and supported project coordination and documentation efforts.

- Prince Kumar : AI Integration Engineer

  Working as the AI Integration Engineer, focusing on the design and optimization of prompts for accurate and efficient complaint processing. He was responsible for refining AI-generated outputs to enhance clarity and reliability, contributed to the design of system data flow and module interactions, and ensured consistent version control and collaboration through diligent maintenance of GitHub repositories.

6. AI USAGE DECLARATION

- Backend API

  The backend API development followed a hybrid approach, combining manual implementation with AI assistance. Core server logic, routing, and integrations were primarily designed and implemented manually, while AI support was leveraged to enhance development efficiency and optimize certain components.

- Frontend UI

  The frontend user interface was developed predominantly through manual effort, ensuring precise control over design, user experience, and responsiveness. This approach allowed for a more tailored and intuitive interaction flow for users.

- AI Processing

  The AI processing module was developed using AI-generated components that were further customized to meet the specific requirements of the system. This includes prompt design, output refinement, and intelligent complaint classification to ensure accuracy and reliability.

- AI Tools Used:

  We have utilized ChatGPT, Groq LLM, Claude, and Gemini for code assistance, AI processing, prompt refinement, and result validation.

- Manual Work:

  The manual development effort focused on React UI implementation, API integration, and effective state management to ensure smooth functionality and user experience.

- Mixed Work:

  The base logic was developed manually and further enhanced using AI to improve routing efficiency and request handling to ensure accuracy and system reliability.

7. PROMPTS LOG

- Frontend

1. I already have the basic express setup, now add middleware so it can read json data from requests properly
2. create a post route where user can send complaint data (text for now, later voice maybe)
3. check if user is even sending valid complaint or just empty string or null (handle that properly)
4. wrap the route logic inside try catch because async errors were crashing earlier
5. integrate groq api here so that complaint can be processed using llm
6. write a helper function that sends user complaint to llm and gets some structured response back
7. store api keys in env file instead of hardcoding (i think this is safer)
8. also handle case where api key is missing or wrong, right now it just breaks
9. now based on llm output try to classify which department this complaint belongs to
10. sometimes llm gives weird or incomplete output so add a fallback just in case
11. implement some routing logic manually also instead of fully depending on ai
12. add location based routing logic (not very accurate yet but basic working)
13. send proper structured response back to frontend so it can display nicely
14. code is getting messy now, break it into smaller functions so its readable
15. response format is inconsistent, fix it so frontend doesnt get confused
16. add proper status codes (i forgot this earlier like always)
17. handle routes that dont exist, currently it just shows cannot get error
18. create a centralized error handler instead of repeating same thing everywhere
19. add some logging so i can debug easily when something fails
20. fix async await usage because sometimes promises are not resolving properly
21. sanitize input a bit, not doing heavy validation but basic cleanup atleast
22. test api with random inputs and edge cases, found few bugs here
23. improve routing logic little more because results were not very accurate
24. use port from env instead of hardcoding 3000 (small thing but important)
25. if routing or geo logic fails just return a default response instead of crashing

- Backend

1. import required packages like express, dotenv etc (forgot env earlier so adding now)
2. configure env file properly so env variables actually load (was not working before)
3. set up port using process.env.PORT with fallback just in case
4. add middleware for json parsing because req.body was coming undefined
5. also enable cors since frontend was getting blocked sometimes

6. run server once and check if it starts without errors
7. connect complaint routes from app.js or routes file into this server
8. double check route path because earlier it was mismatching
9. test endpoint using postman to see if request is reaching backend
10. add temporary console logs to track incoming requests for debugging
11. handle server start errors like port already in use (happened once)
12. add basic error handling so server doesnt crash fully
13. clean up imports and structure a bit, code was getting messy
14. fix small typo in variable names causing undefined issue
15. verify env variables (like api keys) are actually being used correctly
16. test full flow from frontend to backend and check response consistency
17. remove extra debug logs and do final cleanup so server runs properly

8. MILESTONES & TIMELINE

- Release 1:

  The first release focused on identifying the core problem and designing an initial solution framework. It included the development of a basic frontend UI to enable user interaction, along with setting up the backend infrastructure. Additionally, initial AI integration was implemented to begin processing and analyzing user complaints.

- Release 2: (yet to do)

  The second release will focus on completing the full feature set of the application, ensuring all planned functionalities are fully developed and integrated. It will include database integration for storing and tracking complaints, along with performance optimization to improve system efficiency and scalability. Additionally, this phase will involve deploying the application using appropriate DevOps and cloud infrastructure for reliable and production-ready operation.

9. GOALS FOR RELEASE 2

The second release aims to implement the complete complaint submission workflow, ensuring a seamless end-to-end experience for users. A robust REST API with at least five endpoints will be developed to support various functionalities. The system will integrate a database to enable efficient storage and tracking of complaints. Efforts will also be directed toward improving the accuracy of AI-based classification, along with introducing authentication mechanisms if required for enhanced security. Additionally, the application will be deployed on a cloud platform to ensure scalability and reliability, while strengthening support for multilingual inputs to make the system more inclusive.

- Challenges:

  Key challenges anticipated in this phase include maintaining consistency in AI-generated responses and ensuring accurate mapping of complaints to the appropriate departments.

- Mitigation:

  To address these challenges, prompt refinement techniques will be applied to improve AI output quality, and extensive testing with diverse inputs will be conducted to enhance system robustness and accuracy.